

Codex 文件读取工具设计调研

Shell-as-Read-Primitive: 从 Tool Schema、Safety Classifier 到 System Prompt

核心结论 Codex 当前没有面向模型的通用本地 Read(path, offset, limit) 工具。常规文件发现、搜索与读取主要由 shell_command / exec_command 执行 rg、cat、sed、head、tail、nl、git show 等命令完成；写入则倾向使用结构化 apply_patch。

调研对象：openai/codex 主仓库 current main；检索日期：2026-08-02。

报告重点：Read 能力边界、safe command 判定、Prompt 引导、输出控制、跨平台差异，以及对自研 Agent Harness 的启示。

目录

1. 执行摘要与核心判断
 2. 模型真正看见的工具面
 3. Codex 的 Read 生命周期
 4. Unix/Linux/macOS：哪些 shell command 被视为 known-safe
 5. Windows：PowerShell Read safelist
 6. System Prompt 如何引导模型读取代码
 7. 输出截断与并行读取
 8. 为什么 Read 走 Shell，而 Write 走 apply_patch
 9. 与传统 Read/Grep/Glob Tool 设计的对比
 10. 对自研 Agent Harness 的工程建议
- 附录 A. 与 Read 直接相关的原文提示词
- 附录 B. 可直接复用的完整重建版 System Prompt
- 附录 C. 源码索引

1. 执行摘要与核心判断

如果把“Read 工具”定义为 Claude Code 一类显式的结构化接口，例如 `Read(file_path, offset, limit)`，那么 Codex 当前核心工具面中确实没有对称的本地文本 Read Tool。模型的 workspace 读取行为主要被折叠到通用 shell 执行接口中。[S1][S2]

一句话模型 Codex = Shell for Discovery / Search / Read + apply_patch for Structured Mutation。

这不意味着 Codex 内部没有文件读取 API。仓库中存在执行层、RPC 层和 MCP Resource 层的 read 能力；关键区别在于：这些内部能力不等于“模型可见的 Read Tool”。研究 Agent Harness 时必须区分 model-facing tool surface 与 runtime/filesystem implementation。

- 模型可见：shell_command 或 exec_command + write_stdin，用于执行 shell/PTY 命令。[S2]
- 编辑可见：apply_patch 是独立的 freeform custom tool，并由 grammar 约束 patch 语言。[S5]
- MCP 例外：存在 ReadMcpResourceHandler，但它读取的是 MCP Resource，不是普通 workspace 文件。[S1]
- 安全层：Codex 对 shell invocation 做“是否 known-safe / 足够只读”的静态分类，而不是简单按可执行文件名白名单放行。[S3][S4]
- Prompt 层：模型 instructions 明确把 cat、rg、sed、ls、git show、nl、wc 称为 file reads，并要求尽可能并行读取。[S6]

2. 模型真正看见的工具面

2.1 shell_command / exec_command

传统 shell_command 接受 command、workdir、timeout_ms 等参数；新版 exec_command 接受 cmd、workdir、tty、yield_time_ms、max_output_tokens 等参数。exec_command 的输出 schema 还显式包含 original_token_count 与可能已经被截断的 output。[S2]

```
shell_command({
  "command": "sed -n '120,220p' codex-rs/core/src/tools/spec_plan.rs",
  "workdir": "/repo/codex"
})
```

因此模型不是调用 read_file("...")，而是提交“如何观察 workspace”的 shell program。

2.2 apply_patch

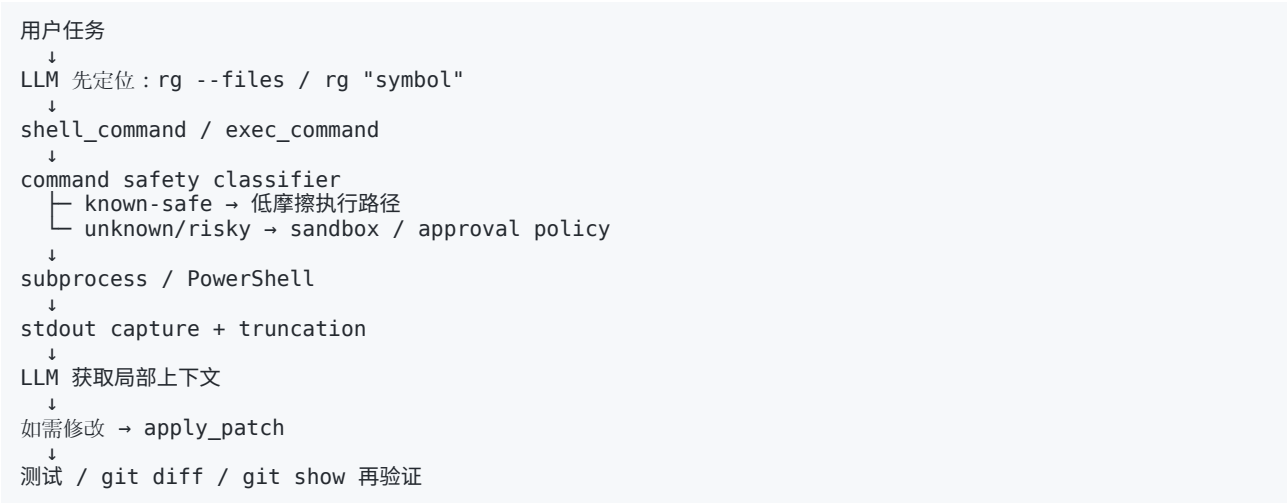
apply_patch 是单独的 freeform tool，描述为用于编辑文件，并使用 Lark grammar 约束输入。它不是 Read 的对称实现，而是专门提升出来的 mutation primitive。[S5]

```
*** Begin Patch
*** Update File: src/parser.rs
@@
- old implementation
+ new implementation
*** End Patch
```

2.3 为什么 “ReadMcpResource” 不能算普通 Read

Tool Registry 中确实存在 ReadMcpResourceHandler，但它对应 MCP Resource 语义。它证明 Codex 能暴露结构化“读取资源”的工具，却没有把相同抽象用于普通 workspace 文本文件。[S1]

3. Codex 的 Read 生命周期



这里最重要的不是“cat 能读文件”这个事实，而是 Codex 在 Prompt、Safety、Sandbox、Output Budget 四层共同把 Shell 塑造成了可控的 Read substrate。

4. Unix/Linux/macOS: known-safe shell command

is_known_safe_command() 的语义不是“只有这些命令才能执行”，而是“这些 invocation 已经可以被静态判断为足够低风险/只读，从而进入 auto-approval 友好的路径”。源码甚至将 Windows 对应逻辑称为 read-only enough to be auto-approved。[S3][S4]

4.1 直接 safelist

命令	Read 角色	说明
cat	读取完整文件	典型 ReadFile
head / tail	读取头部/尾部	适合日志、超长文件
nl	带行号读取	便于后续 patch 与定位
grep	文本搜索	基础 search primitive
ls	目录观察	相当于 ListDirectory
stat	文件元信息	大小、时间、权限等
wc	计数	常用于 wc -l 判断文件规模
cut / paste / tr / uniq	纯文本变换	可作为只读 pipeline 的后处理
pwd / which / id / whoami / uname	环境观察	不直接读代码，但属于 observation
Linux: tac / numfmt	只读文本工具	平台特定 safelist

源码直接 safelist 还包含 cd、echo、expr、false、rev、seq、true 等。它们并非都属于“文件读取”，但都被当前分类器视为可安全 exec 的低副作用命令。[S3]

4.2 sed：只放行“像 ReadRange 的 sed”

Codex 没有把整个 sed 视为 safe，而是专门识别 sed -n Np 与 sed -n M,Np，并且限制参数形状。这相当于在 shell 语法中识别出一个隐式 ReadRange primitive。[S3]

```
sed -n '120p' src/parser.rs
sed -n '120,220p' src/parser.rs
```

关键边界 sed -i 等可修改文件的模式不会命中这个 known-safe 分支。安全判定看的是 invocation 语义，而不是仅看 executable == sed。

4.3 rg：默认安全，但危险 flag 会退出 safe path

普通 rg 与 rg --files 是核心 exploration primitive。分类器显式拒绝 --pre、--hostname-bin、--search-zip、-z 等选项进入 known-safe，因为这些选项可能执行额外程序或调用解压工具。[S3]

Invocation	分类直觉
rg "ToolRegistry" codex-rs/core	安全搜索
rg --files	安全文件发现
rg --pre <cmd> ...	不再 known-safe：可执行任意 preprocessor
rg --search-zip / -z ...	不再 known-safe：可能调用外部解压工具

4.4 find：可以查找，但不能带副作用 option

find 会被检查参数；-exec、-execdir、-ok、-okdir、-delete、-fls、-fprint、-fprint0、-fprintf 会让它退出 known-safe，因为这些 option 可以执行命令、删除文件或写文件。[S3]

4.5 git：被当作“版本化读取接口”

Codex 显式认可 git status / log / diff / show / branch 的只读用法。它们让模型观察的不只是当前文件，还包括历史版本、工作区差异与分支状态。[S3]

命令	等价 Agent 能力
git status	工作区状态观察
git diff	ReadDiff
git show HEAD:path	ReadHistoricalFile
git log -p	历史 patch / provenance
git branch --show-current	branch metadata

同时，-C、-c、--git-dir、--work-tree、--paginate、--output、--ext-diff、--textconv、--exec 等 global/subcommand option 会触发更保守处理。[S3]

4.6 组合命令：只读 pipeline 可以整体判 safe

对 `bash -lc "..."`，Codex 会解析由 plain commands 组成的脚本，并仅保守允许 `&&`、`||`、`;`、`|` 这类 operator；只有每个子命令都通过 safe classifier，整体才可视为 safe。[S3]

```
rg -n 'ToolRegistry' codex-rs/core | head -n 30
nl -ba src/parser.rs | sed -n '100,180p'
```

这使 Shell 能表达传统 Read Tool 中的 offset/limit/line-number 等组合能力，而无需增加新的 JSON schema。

5. Windows: PowerShell Read safelist

Windows 路径更保守：只有可以解析为 PowerShell 且命中明确 read-only safelist 的 invocation 才算 safe；直接 CMD 或未知 cmdlet 不会被默认为 safe。[S4]

PowerShell / Alias	角色
Get-ChildItem / gci / dir / ls	目录遍历
Get-Content / gc / cat / type	文件内容读取
Select-String / sls / findstr	文本搜索
Measure-Object / measure	计数/测量
Get-Location / gl / pwd	当前目录
Test-Path / tp	路径存在性
Resolve-Path / rvpa	路径解析
Select-Object / select	pipeline 选择/截断
Get-Item	文件/对象元信息
git	复用 Git 只读分类
rg	复用 ripgrep flag 安全检查

分类器还显式拒绝 Set-Content、Add-Content、Out-File、New-Item、Remove-Item、Move-Item、Copy-Item、Rename-Item、Start-Process、Stop-Process 等明显有副作用的 cmdlet。[S4]

6. System Prompt 如何引导模型读取代码

Codex 并非只在 runtime 层允许 shell read；当前 models.json 中多组模型 instructions 都直接塑造 exploration 策略。[S6]

6.1 搜索优先 rg / rg --files

Prompt 明确要求搜索文本优先 rg、搜索文件优先 rg --files；rg 不可用时才 fallback。这把 repo exploration 的第一步固定成高性能搜索，而不是先 cat 大量文件。

6.2 把 shell 命令明确称为 file reads

更关键的是，Prompt 把 cat、rg、sed、ls、git show、nl、wc 直接列为 file reads，并鼓励尽可能并行调用。这证明 shell-based read 是产品化的预期行为，而非模型偶然习惯。[S6]

```
定位：    rg --files
搜索：    rg -n "ToolRegistry" codex-rs/core
局部读：  sed -n '120,220p' target.rs
行：      nl -ba target.rs | sed -n '120,220p'
史：      git show HEAD~1:target.rs
模判：    wc -l target.rs
```

6.3 不鼓励 Python 充当临时 Read Tool

当前 instructions 中存在两类约束：不要为了输出大段文件内容写 Python 脚本；当简单 shell command 足够时，不要使用 Python 读写文件。[S6] 这进一步说明 Codex 希望优先复用 OS/CLI primitives。

6.4 明确区分“读”和“写”

Prompt 同时强调：手工编辑用 apply_patch，不要用 cat 或其他 shell write tricks 创建/编辑文件。[S6] 因此同一个 cat 在语义上被明确放在 observation 一侧，而 mutation 被引导到 apply_patch。

6.5 Tool Description 也承担行为约束

shell_command 的工具描述要求模型设置 workdir，并尽量不要通过 cd 改变目录；exec_command 则把 workdir、PTY、yield_time 与 max_output_tokens 做成显式参数。[S2] 这是一种“把可结构化的环境状态留在 Tool Schema，把实际工作留给 shell string”的折中。

7. 输出截断与并行读取

7.1 为什么没有 Read(offset, limit) 仍然可控

新版 exec_command 的 max_output_tokens 默认描述为 10,000 tokens，并在 output schema 中返回 original_token_count 和 possibly truncated output。[S2] 所以即使模型误用 cat 读取巨大文件，runtime 仍有输出预算保护。

两层节流 模型侧：rg → sed/head/tail 局部读取；Runtime 侧：stdout token budget + truncation。

7.2 Prompt 为什么强调 parallel file reads

Coding Agent 的 exploration latency 很大一部分来自 LLM ↔ Tool 串行往返，而不是本地 cat/rg 本身。Codex 因此鼓励独立读取并行化，例如同时查看 parser、lexer、token 三个文件，而不是三轮串行 ReAct。[S6]

```
LLM ———┐ cat parser.rs
          ├── cat lexer.rs
          └── cat token.rs
              ↓
          一轮聚合上下文
```

8. 为什么 Read 走 Shell，而 Write 走 apply_patch

这是一种有意的非对称设计。Read/observe 的副作用低、CLI 生态成熟、组合能力强；Write/mutation 则需要 Harness 更强的可理解性、路径控制、审批和 diff 展示。

维度	Shell Read	apply_patch Write
表达力	高：可组合 rg/sed/git/nl/wc	专注文件变更
副作用	通常低，可静态识别	天然有副作用
结构化程度	低：command string	高：文件级 patch grammar
审批/审计	靠 classifier + sandbox	更易知道改了哪些 path
上下文效率	可先搜索再局部读	只传递 diff，而非整文件重写
生态复用	直接复用 Unix/PowerShell/Git	Harness 自己维护

因此“Codex 没有 Read Tool”不应被理解成能力缺失，而应理解成抽象边界选择：Read 被下沉到 shell language，Mutation 被提升到 Harness-aware primitive。

9. 与传统 Read/Grep/Glob Tool 设计的对比

传统 Agent Tool	Codex 常见 shell 表达
Glob / ListFiles	rg --files / find / ls
Grep / SearchText	rg pattern
ReadFile	cat file
ReadRange(offset, limit)	sed -n M,Np
ReadWithLineNumbers	nl -ba file
ReadTail	tail -n N file
ReadMetadata	stat / wc
ReadHistoricalFile	git show REV:path
ReadDiff	git diff

传统工具的优势是 schema 清晰、权限语义直接、跨平台一致、容易做 token-level 截断；Shell 的优势是工具数少、组合能力强、模型训练数据丰富、可以自然扩展到 Git/编译器/测试/包管理器。Codex 选择后者，但用 classifier 和 sandbox 补足安全边界。

10. 对自研 Agent Harness 的工程建议

如果你希望借鉴 Codex，而不是机械复制“只给 Bash”，建议至少实现以下四层：

- Prompt policy：规定 search-first、局部读取、并行读取、禁止用脚本无意义 dump 大文件。
- Semantic command classifier：按 executable + args + pipeline 语义判断 read-only，而不是只维护 executable whitelist。
- Sandbox / approval：known-safe 只是低风险信号，不应该替代 filesystem/network sandbox。

- Output budget: 统一截断 stdout, 返回原始 token/line count, 让模型知道结果可能不完整。

如果第一版追求实现简单, 可以采用一个“小 Codex”方案:

```
builtin tools:
- shell(command, workdir, timeout_ms, max_output_tokens)
- apply_patch(patch)

read policy:
- prefer rg / rg --files
- read slices with sed -n / head / tail
- use nl when line numbers matter
- use git show/diff for version-aware reads
- parallelize independent observations

safety:
- static safe classifier
- workspace sandbox
- approval escalation
- stdout truncation
```

不建议 仅仅暴露一个 unrestricted Bash, 然后把“安全”完全交给模型自觉。Codex 的效果来自 Prompt + classifier + sandbox + output policy 的组合, 而不是 Bash 本身。

附录 A. 与 Read 直接相关的原文提示词

说明: 以下只收录 openai/codex 当前 models.json 与 shell tool schema 中“直接影响文件读取策略”的相关原文段落, 不复制与本报告无关的整份长 System Prompt。来源见 [S2][S6]。

A.1 搜索与文件读取策略 (models.json)

When searching for text or files, prefer using `rg` or `rg --files` respectively because `rg` is much faster than alternatives like `grep`. (If the `rg` command is not found, then use alternatives.)

Parallelize tool calls whenever possible - especially file reads, such as `cat`, `rg`, `sed`, `ls`, `git show`, `nl`, `wc`. Use `multi\_tool\_use.parallel` to parallelize tool calls and only this.

A.2 避免 Python dump 文件 (models.json)

Do not use python scripts to attempt to output larger chunks of a file.

Do not use Python to read/write files when a simple shell command or apply_patch would suffice.

A.3 读写边界 (models.json)

Always use apply_patch for manual code edits. Do not use cat or any other commands when creating or editing files.

A.4 shell_command Tool Description

Runs a shell command and returns its output.
- Always set the `workdir` param when using the shell_command function. Do not use `cd` unless absolutely necessary.

A.5 exec_command 与输出预算 (Tool Schema 摘要)

cmd: Shell command to execute.
workdir: Working directory for the command. Defaults to the turn cwd.
max_output_tokens: Output token budget. Defaults to 10000 tokens; larger requests may be capped by policy.
output: Command output text, possibly truncated.
original_token_count: Approximate token count before output truncation.

附录 B. 可直接复用的完整重建版 System Prompt

下面这份 Prompt 是基于本次源码调研重建的“Codex-style Read Policy”，用于你自己的 Agent Harness。它不是 openai/codex 原文逐字复制，而是把仓库中的工具语义、安全边界与读取策略整合成一个完整、可直接使用的版本。

Workspace Exploration and File Read Policy

You are a coding agent operating inside a user workspace. Build context from the codebase before making assumptions.

Search first

- When searching for text, prefer `rg` because it is fast and returns focused results.
- When discovering files, prefer `rg --files`. If `rg` is unavailable, fall back to appropriate platform tools.
- Avoid dumping an entire repository or reading many full files before you know which regions matter.

Reading files with shell primitives

Use ordinary read-only shell commands as the primary file-read interface:

- `cat <file>` for small files that need full context.
- `head -n N <file>` or `tail -n N <file>` for file boundaries and logs.
- `sed -n 'M,Np' <file>` for focused line ranges.
- `nl -ba <file>` when stable line numbers help reasoning or later edits.
- `wc -l <file>` or `stat <file>` before reading very large files.
- `rg -n <pattern> <path>` to locate symbols or references before reading surrounding ranges.
- `git diff`, `git show`, `git log -p`, and read-only `git branch` queries when version history or working-tree state matters.

Minimize context waste

- Prefer search -> targeted range read -> wider read only when necessary.
- Do not write Python scripts merely to print file contents when a simple shell command is enough.
- If command output is truncated, narrow the query or read smaller ranges instead of repeatedly requesting the same large output.

Parallel reads

- Parallelize independent searches and file reads whenever the tool runtime supports parallel calls.
- Good candidates include independent `cat`, `rg`, `sed`, `ls`, `git show`, `nl`, and `wc` calls.
- Do not create artificial shell command chains merely to simulate parallelism if the harness already supports parallel tool calls.

Working directory

- Set the tool's `workdir` explicitly when possible.
- Prefer the structured `workdir` parameter over embedding `cd ... &&` into the shell command.
- Keep commands non-interactive unless interaction is required.

Read versus write

- Treat shell-based file reads as observation operations.
- For manual file edits, use the structured patch/edit tool provided by the harness.
- Do not create or modify files with `cat >`, `echo >`, `sed -i`, or similar shell write tricks when the structured edit tool is available.
- Formatting tools and generated outputs may be exceptions when they are the intended authoritative mechanism.

Safety

- Prefer commands whose effects are clearly read-only.
- Avoid shell options that execute arbitrary subprocesses or write/delete files during what should be an observation step.
- Examples of suspicious read-like invocations include `find -exec`, `find -delete`, `rg --pre`, output redirection, or git options that write to files or invoke external diff/textconv programs.
- If a required command needs additional filesystem or network permissions, request them through the harness permission mechanism instead of trying to bypass the sandbox.

Validation loop

For code changes, use this sequence when practical:

1. Locate relevant files and symbols.
2. Read the smallest sufficient regions.
3. Apply a structured patch.
4. Re-read the changed region or inspect `git diff`.
5. Run focused tests or validation commands.

The goal is not to minimize shell usage. The goal is to use shell as a composable observation language while keeping mutations structured, auditable, and sandbox-aware.

附录 C. 源码索引

[S1] Tool Registry / model-visible specs

https://github.com/openai/codex/blob/main/codex-rs/core/src/tools/spec_plan.rs

[S2] shell_command / exec_command Tool Schema

https://github.com/openai/codex/blob/main/codex-rs/core/src/tools/handlers/shell_spec.rs

[S3] Unix/Linux/macOS safe command classifier

https://github.com/openai/codex/blob/main/codex-rs/shell-command/src/command_safety/is_safe_command.rs

[S4] Windows PowerShell safe command classifier

https://github.com/openai/codex/blob/main/codex-rs/shell-command/src/command_safety/windows_safe_commands.rs

[S5] apply_patch freeform Tool Schema

https://github.com/openai/codex/blob/main/codex-rs/core/src/tools/handlers/apply_patch_spec.rs

[S6] Model instructions / models.json

<https://github.com/openai/codex/blob/main/codex-rs/models-manager/models.json>

注：Codex 主分支变化较快。本报告反映 2026-08-02 检索时的 current main；后续版本可能调整模型 prompt、shell tool type 或 safety classifier。